Qwen Forecast

Do dado à previsão auditável

Guia didático da implementação · 5 de outubro de 2026

Este documento acompanha o código reorganizado deste projeto. Ele explica como transformar uma série de produção de água em um prompt, enviar esse prompt ao Qwen pelo Ollama e avaliar as previsões junto com Chronos e TimesFM.

> **O que o Qwen faz aqui?** Recebe números históricos como texto e gera os próximos sete valores. O projeto não treina nem ajusta os pesos do Qwen. A expressão *qwen_forecast* designa essa rotina de inferência com o modelo local **qwen3:1.7b**.

Os exemplos de código são extraídos dos arquivos reais. Caminhos são relativos à raiz do repositório. Imports aparecem apenas quando ajudam a entender uma dependência; o foco está em contratos, parâmetros, fluxo e prevenção de vazamento.

Roteiro de leitura

1. Arquitetura e protocolo temporal.
2. Leitura, validação e janelas.
3. Construção de prompts e exemplos few-shot.
4. Dataclasses, argumentos e interfaces.
5. HTTP, saída estruturada, validação e retentativas.
6. Orquestração, cache, métricas e resultados.
7. Como executar, testar e estender o projeto.

Leia primeiro os capítulos de prompts e Ollama se já conhecer validação temporal. Para reproduzir a execução, use o capítulo de comandos e a configuração em `configs/benchmark.json`.

1 · Arquitetura e responsabilidades

A organização segue as camadas solicitadas. Cada camada tem um contrato: dados entram validados, modelos recebem somente história e a avaliação acessa os alvos depois da previsão. Isso reduz o risco de um adaptador receber uma coluna futura por acidente.

Camada / arquivo	Responsabilidade
data/ingestion/files.py	read_series lê o CSV; write_json grava artefatos completos.
data/validation/schema.py	validate_series valida datas e demanda.
data/preprocessing/	daily_grid expõe lacunas; fill_past_only preenche causalmente; generate_prompts monta o texto.
data/features/	future_dates constrói datas conhecidas; demand_only explicita a seleção univariada.
evaluation/cross_validation.py	ForecastWindow representa recortes; build_windows define origens comuns.
models/promptcast.py	OllamaConfig, QwenForecaster e funções de contrato HTTP/JSON.
models/chronos.py e timesfm.py	Adaptam bibliotecas especializadas à mesma assinatura predict.
pipelines/	data_pipeline prepara; forecasting_pipeline executa; compare_models confere e compara.
training/train.py	Reserva uma extensão temporal de treino/validação; não faz fit neste experimento.

Fluxo principal

CSV → validação → grade diária → preenchimento causal → janelas

↓

história → prompt → Ollama → resposta → validação → previsão

↓

alvo observado + previsão → CSV por modelo → métricas comparáveis

O adaptador Qwen não conhece o dataframe completo. A função `build_prompt` também recebe apenas história. O comparador trabalha depois das inferências e exige as mesmas datas para os três modelos.

Arquivos vazios de API, banco de dados e Moirai não foram criados: não existe uma implementação ativa desses recursos. Os scripts antigos de execução foram substituídos. As fontes CSV foram preservadas, e o notebook foi convertido em análise somente de leitura.

2 · Protocolo temporal e limites da garantia

Conjunto	Contexto	Horizonte / passo	Janelas
Joinville	365 dias	7 / 7 dias	54
Guaratuba	565 dias	7 / 7 dias	104

Uma origem `0` representa o primeiro dia a prever. A história ocupa `[0-contexto, 0)`, e o alvo ocupa `[0, 0+7)`. O parêntese significa que o limite final é exclusivo. Nenhum valor do alvo da janela entra na chamada ao modelo.

A primeira origem é o maior entre o tamanho do contexto e o índice que inicia os dois últimos anos da série. Em Joinville, isso produz 16/06/2024; em Guaratuba, 01/01/2018. O passo de sete evita sobreposição entre os alvos. Uma sobra final inferior a sete dias não gera janela incompleta.

Contexto não é treinamento de pesos

Em uma avaliação de origens móveis, um valor observado na primeira semana de teste pode entrar na história de uma previsão semanas depois. Isso representa informação já disponível naquela nova data. Neste projeto ele é usado somente como contexto de inferência; nenhum otimizador, ajuste de pesos ou escolha de hiperparâmetros usa esses testes.

Exemplos few-shot têm uma regra ainda mais restrita: ficam congelados no prefixo anterior ao primeiro alvo avaliado. Assim, nenhum teste vira resposta de demonstração. O benchmark executado usa zero-shot, sem essas demonstrações.

> Clima observado no futuro não pode ser renomeado como previsão meteorológica. Sem data de emissão que prove sua disponibilidade na origem, ele é excluído. Também excluímos colunas derivadas, cuja origem causal não está documentada.

A garantia cobre a preparação e execução locais. Os CSVs recebidos já estavam processados: não há aqui evidência suficiente para auditar toda a transformação anterior. Tampouco este repositório permite provar ausência das séries nos corpora externos de pré-treino dos modelos.

Não há validação para ajuste neste experimento. Para uma extensão de treino, `training_partition(first_test_origin, validation_fraction=0.2)` delega a `validation_split` e separa os últimos 20% do prefixo de treino, mantendo o teste fora de ambos.

3 · Ler e preparar dados sem olhar à frente

read_series e validate_series

`read_series(path)` lê o CSV e chama `validate_series(df)`. Esta mantém somente `timestamp` e `water_produced`, converte tipos, ordena cronologicamente e rejeita datas vazias/duplicadas, horários intradiários e demanda inválida. Duplicatas geram erro em vez de uma escolha arbitrária de qual observação conservar.

O retorno é um novo dataframe. `.copy()` torna explícita a intenção de não modificar a fonte. `errors="raise"` impede conversões silenciosas de entradas ruins. `np.isfinite` detecta tanto NaN quanto infinito.

daily_grid e fill_past_only

`daily_grid(df)` usa a data como índice e `asfreq("D")` materializa todos os dias entre o primeiro e o último. A coluna `observed` é marcada antes de qualquer preenchimento. Em Guaratuba falta 29/02/2016: esse dia passa a existir como lacuna explícita.

```
def fill_past_only(df: pd.DataFrame) -> pd.DataFrame:
    result = df.copy()
    result['water_produced'] = result.water_produced.ffill()
    if result.water_produced.isna().any():
        raise ValueError('Cannot fill initial missing demand without past data')
    return result
```

`ffill()` usa a última observação anterior. Não usa a observação seguinte, como ocorreria com várias formas de interpolação. A marca `observed=False` permanece, permitindo ao construtor de janelas recusar alvos preenchidos.

prepare_dataset

```
def prepare_dataset(path, context, *, horizon=7, step=7, evaluation_years=2):
    df = fill_past_only(daily_grid(read_series(path)))
    windows = build_windows(df, context, horizon, step, evaluation_years)
    return df, windows
```

O retorno é uma tupla `(df, windows)`. O dataframe preparado conserva a marca de observação; a lista de janelas contém somente recortes elegíveis. Os parâmetros depois de `*` são obrigatoriamente nomeados: isso evita confundir horizonte, passo e anos de avaliação quando todos são números.

4 · Janelas e @dataclass

Uma `dataclass` é uma classe que recebe automaticamente métodos comuns, como inicialização, representação textual e comparação por campos. Aqui ela dá nome aos limites temporais em vez de espalhar tuplas de índices sem significado.

```
@dataclass(frozen=True)
class ForecastWindow:
    context_start: int
    origin: int
    horizon: int

    @property
    def history(self) -> slice:
        return slice(self.context_start, self.origin)

    @property
    def target(self) -> slice:
        return slice(self.origin, self.origin + self.horizon)
```

Ao escrever `ForecastWindow(0, 365, 7)`, Python cria uma instância com `context_start=0`, `origin=365` e `horizon=7`. O decorador `@dataclass(frozen=True)` impede reatribuir esses campos, ajudando a manter estável o protocolo durante a execução. Essa imutabilidade é superficial: em outras classes, uma lista contida em um campo ainda poderia ser mutável.

`@property` transforma um método sem parâmetros extras em acesso semelhante a atributo. `window.history` executa o método e retorna um `slice`; não é preciso escrever parênteses. `df.iloc[window.history]` seleciona posições, não rótulos de índice.

Como build_windows usa a classe

`build_windows(df, context, horizon=7, step=7, evaluation_years=2)` valida limites positivos, calcula a primeira origem elegível e percorre `range(first, len(df)-horizon+1, step)`. O `+1` inclui a última origem que ainda tem sete alvos completos.

A expressão `df.iloc[origin:origin+horizon].observed.all()` exige observação real em todo o alvo. Para cada origem válida, cria uma `ForecastWindow(origin-context, origin, horizon)`. Sem janelas válidas, lança erro; não devolve um benchmark aparentemente bem-sucedido e vazio.

Outras dataclasses

`Demonstration` guarda história, resposta e fim do alvo de um exemplo histórico em tuplas. `OllamaConfig` reúne limites e opções de inferência. `dataclasses.asdict(config)` transforma os campos em dicionário serializável para auditoria e cálculo de hash.

5 · Transformando história em prompt

O arquivo central é `src/data/preprocessing/generate_prompts.py`. O prompt da comparação contém somente demanda, instruções, datas e formato de saída. Datas futuras são determinísticas, calculadas a partir do último dia histórico.

Função / parâmetro	Uso concreto
format_values(values)	Formata cada valor com duas casas decimais e une por vírgula, sem separador de milhares.
future_dates(last_observed, horizon)	Gera os dias seguintes com pd.date_range; não consulta a tabela de teste.
build_prompt(history, horizon, *, demonstrations=None)	Valida o histórico, descreve a tarefa, insere exemplos opcionais e serializa a sequência.
history	Dataframe já recortado com timestamp e water_produced; nunca o dataset completo.
horizon	Número exato de valores exigidos na saída; sete no benchmark.
demonstrations	Lista opcional de exemplos históricos; None equivale a zero-shot.

A ordem dos blocos

1. Tarefa e unidade: prever produção diária de água em metros cúbicos.
2. Orientação para inferir nível, tendência e sazonalidade.
3. Contrato: objeto JSON com `predictions` e sete números não negativos.
4. Exemplos históricos, se solicitados.
5. Quantidade de dias, datas de início/fim e sequência do mais antigo ao mais recente.
6. Datas previstas e lembrete do esquema de saída.

A sequência compacta reduz o custo de tokens comparada a repetir uma descrição em cada dia. A ordem ainda é inequívoca: há uma grade diária, um intervalo conhecido e a contagem de valores. Arredondar o texto para duas casas decimais é uma diferença de representação em relação às baselines, que recebem os floats originais; isso está registrado, não oculto.

```
history = df.iloc[window.history][["timestamp", "water_produced"]].copy()
prompt = build_prompt(history, window.horizon, demonstrations=[])
# Os alvos só serão acessados após model.predict(...).
```

A função retorna uma string. Ela não faz HTTP, não lê arquivos de resultado e não calcula métricas. Essa separação permite testar que alterar alvos futuros não altera o prompt.

6 · Few-shot sem exemplos vindos do teste

A função `select_demonstrations` preserva duas estratégias úteis da implementação anterior. Elas estão disponíveis para estudos futuros, mas não foram usadas para escolher a configuração a partir dos resultados atuais.

Parâmetro	Significado
training_values	Prefixo congelado antes da primeira origem de teste, passado já recortado pelo pipeline.
history	História da origem atual; define largura e vetor de consulta para similaridade.
horizon	Tamanho da resposta de cada exemplo.
strategy	zero, few_random ou few_similar.
count=3	Quantidade exigida de exemplos; dados insuficientes geram erro.
seed=42	Semente do gerador local usado na amostragem aleatória.

Candidatos

Se `width=len(history)` e há `N` observações no prefixo, os candidatos são origens entre `width` e `N-horizon`, inclusive. Para uma origem candidata `o`, a entrada é `values[o-width:o]` e a resposta é `values[o:o+horizon]`. A resposta inteira deve terminar dentro do prefixo.

few_random

`np.random.default_rng(seed).choice(..., replace=False)` escolhe índices sem repetir o mesmo candidato. Usar um gerador local evita alterar aleatoriedade global de outras partes do programa. Os exemplos escolhidos são organizados cronologicamente antes de entrar no texto.

few_similar

A função interna `standardized(x)` centraliza e divide pelo desvio padrão da própria entrada. Um piso de `1e-8` evita divisão por zero. Compara-se a média das diferenças quadráticas entre entradas padronizadas; os menores valores são selecionados por `np.argsort` estável. A resposta de cada candidato não participa dessa distância.

Essa distância favorece formato temporal, não necessariamente nível absoluto. Os alvos dos exemplos só são anexados depois da escolha. A função não garante exemplos disjuntos entre si, mas garante candidatos distintos; esse é um detalhe metodológico relevante em uma futura análise few-shot.

> Contexto longo + três exemplos pode superar 8.192 tokens. O adaptador rejeita prompts que não cabem no orçamento conservador. Aumente explicitamente `num_ctx` e avalie memória antes de comparar uma nova configuração; não encurte o contexto de uma cidade silenciosamente.

7 · args, *args, **kwargs e argumentos nomeados

São conceitos relacionados, mas diferentes. O objeto chamado `args` no programa vem do parser da linha de comando. Ele não é a mesma coisa que o padrão `*args` de uma assinatura Python.

```
parser.add_argument("--model", choices=["qwen", "chronos", "timesfm"], default="qwen")
args = parser.parse_args()
# args.model contém a escolha feita na linha de comando.
```

`parse_args()` devolve um `argparse.Namespace`, um objeto cujos atributos correspondem às opções registradas. `type=Path` converte um argumento textual em caminho; `choices` rejeita uma opção desconhecida antes de iniciar o modelo.

O asterisco sozinho

```
def build_prompt(history, horizon, *, demonstrations=None):
    ...

build_prompt(history, 7, demonstrations=examples)
```

O `*` sozinho não coleta argumentos. Ele exige que os argumentos seguintes sejam passados pelo nome. A chamada `build_prompt(history, 7, examples)` seria inválida. Isso torna uma opção adicional visível e reduz erros de ordem.

*args e **kwargs em uma definição

```
# Exemplo didático; não é necessário no pipeline atual.
def encaminhar(funcao, *args, **kwargs):
    return funcao(*args, **kwargs)

encaminhar(build_prompt, history, 7, demonstrations=examples)
```

Na definição, `*args` reúne argumentos posicionais extras em uma tupla. `**kwargs` reúne argumentos nomeados extras em um dicionário. Os nomes args e kwargs são convencionais: o que determina o comportamento são os asteriscos. Na chamada, os mesmos símbolos fazem a operação inversa, desempacotando tupla e dicionário.

** usado de verdade no projeto

```
OllamaConfig(**config["qwen"], seed=config["seed"])
# Equivale a passar model=..., base_url=..., temperature=..., etc.
```

As chaves do dicionário precisam coincidir com os nomes aceitos pelo construtor. Chave desconhecida ou argumento repetido gera `TypeError`. Não há função genérica que engula qualquer opção: o construtor tipado ajuda a detectar erros de configuração.

`dict(common, **deterministic_metrics(...))` também desempacota um dicionário, desta vez para compor uma linha do relatório. As anotações como `list[float]` explicam o contrato, mas não validam valores sozinhas: por isso há checagens explícitas no parser.

8 · OllamaConfig e a identidade do modelo

Ollama é o servidor local que carrega e executa o Qwen. Python atua como cliente HTTP. Não é preciso adicionar o SDK Ollama ao ambiente: a biblioteca padrão fornece a comunicação necessária.

Campo	Valor da execução	Efeito
model	qwen3:1.7b	Seleciona o modelo instalado.
base_url	http://127.0.0.1:11434	Endereço do servidor local.
temperature	0	Reduz aleatoriedade da geração; não garante reprodutibilidade bit a bit entre hardwares.
seed	42	Semente enviada ao runtime.
num_ctx	8192	Capacidade de contexto em tokens.
num_predict	256	Limite de tokens gerados, suficiente para sete números.
timeout	600 s	Prazo da operação HTTP; não é o tempo esperado por previsão.
retries	2	Até duas novas tentativas além da primeira.

`OllamaConfig.__post_init__` roda automaticamente após o inicializador gerado pela dataclass. Ele rejeita limites não positivos e quantidade negativa de retentativas. `frozen=True` evita que outra função mude uma configuração no meio da execução.

QwenForecaster.__init__

O construtor recebe uma instância de configuração e consulta `/api/tags`. Ele procura o nome exato e salva seu `digest`, além de `asdict(config)`, em `self.identity`. Se o modelo não está instalado, falha imediatamente. O programa não troca por outro modelo automaticamente.

`self` é a própria instância. Atributos como `self.config` e `self.last_audit` mantêm estado entre chamadas. Cada nova previsão substitui a auditoria anterior por uma estrutura referente àquela janela.

Interface comum

Os três adaptadores possuem `predict(history, horizon, *, prompt=None)` ou a variante que exige prompt no Qwen. O pipeline consegue chamá-los do mesmo modo sem conhecer as APIs específicas. Isso é polimorfismo por contrato: não exige herança, desde que a assinatura e o resultado sejam compatíveis.

9 · HTTP e saída estruturada

post_json(url, payload, timeout)

```
def post_json(url: str, payload: dict, timeout: int) -> dict:
    request = urllib.request.Request(url, data=json.dumps(payload).encode('utf-8'),
                                     headers={'Content-Type': 'application/json'})
    with urllib.request.urlopen(request, timeout=timeout) as response:
        return json.load(response)
```

`json.dumps(payload)` transforma o dicionário em texto JSON; `encode("utf-8")` transforma o texto em bytes para o corpo HTTP. O cabeçalho informa o tipo do corpo. Ao fornecer `data`, a requisição usa POST. O bloco `with` fecha a resposta mesmo que ocorra uma exceção durante a leitura.

`json.load(response)` interpreta o envelope HTTP retornado pelo Ollama. O campo `response` dentro desse envelope ainda é uma string: ela contém o JSON que o modelo gerou e será validada separadamente.

O contrato enviado em format

```
def prediction_schema(horizon: int) -> dict:
    return {'type': 'object', 'properties': {'predictions': {
        'type': 'array', 'items': {'type': 'number', 'minimum': 0},
        'minItems': horizon, 'maxItems': horizon}},
        'required': ['predictions'], 'additionalProperties': False}
```

`minItems` e `maxItems` fixam o tamanho do vetor. `minimum=0` expressa a restrição física do consumo. `required` exige a chave, e `additionalProperties=False` impede chaves extras no contrato. A validação local repete as condições essenciais: não basta confiar no modelo ou no esquema.

Demais campos do payload

`stream=False` pede uma resposta completa, simplificando persistência e parsing. `think=False` desliga a saída de raciocínio do Qwen nesta configuração. `keep_alive="10m"` mantém o modelo carregado entre janelas. `options` contém temperatura, semente e limites; não é um argumento do sistema operacional.

Referências oficiais: Ollama /api/generate e structured outputs. Os exemplos deste manual descrevem a implementação local e não exigem serviços de nuvem.

10 · Validar números, limites e falhas

parse_prediction(text, horizon)

```
def parse_prediction(text: str, horizon: int) -> list[float]:
    """Reject prose, strings, booleans, NaN, missing values and wrong horizons."""
    parsed = json.loads(text)
    if not isinstance(parsed, dict) or set(parsed) != {'predictions'}:
        raise ValueError('Expected exactly the predictions key')
    values = parsed['predictions']
    if not isinstance(values, list) or len(values) != horizon:
        raise ValueError(f'Expected {horizon} predictions')
    if any(type(v) not in (int, float) or not math.isfinite(v) or v < 0 for v in values):
        raise ValueError('Predictions must be finite, nonnegative numbers')
    return [float(v) for v in values]
```

Usamos `type(v) not in (int, float)` intencionalmente: em Python, `bool` é uma subclasse de `int`. Uma validação com apenas `isinstance(True, int)` aceitaria `True` como previsão. Valores textuais como `"12"` também são recusados, evitando correções silenciosas.

`math.isfinite` recusa NaN e infinito mesmo que o decodificador JSON os aceite. O parser não remove cercas Markdown, não extrai números de explicações e não completa posições ausentes. Uma resposta inválida é registrada como falha, não substituída por uma previsão artificial.

QwenForecaster.predict

Antes do HTTP, conta os bytes UTF-8 do prompt e reserva `num_predict + 512` posições para saída e template. É uma estimativa conservadora para a tokenização do Qwen, que pode rejeitar um texto que caberia; o objetivo é impedir truncamento silencioso. A resposta também é verificada quanto a `done`, motivo `length` e contagens de tokens.

O laço executa no máximo `retries+1` tentativas. Cada uma registra resposta bruta ou erro. Em falhas previstas de JSON/validação, conexão ou timeout, espera 1 e depois 2 segundos antes de tentar de novo. Na última tentativa, `raise` propaga o erro original. A temperatura permanece a mesma: uma saída inválida determinística pode repetir-se.

O pipeline captura a falha da janela, grava `status="error"` e continua coletando diagnóstico. Ao final retorna código de saída não zero; o script de comparação não prossegue com resultados incompletos.

11 · Orquestração, cache e auditoria

main e create_model

`main()` interpreta a linha de comando, lê o JSON de configuração e percorre os datasets selecionados. `create_model(name, context, config)` importa apenas o adaptador escolhido. Esse carregamento tardio permite executar Qwen em um ambiente que não tenha as bibliotecas de Chronos ou TimesFM.

run_dataset: ordem que evita vazamento

1. Prepara a série e as janelas comuns.
2. Constrói o modelo e a procedência da execução.
3. Congela `training_values` no prefixo anterior ao primeiro teste.
4. Recorta somente a história da janela.
5. Para Qwen, escolhe exemplos, constrói e salva o prompt.
6. Verifica se uma resposta válida pertence exatamente à requisição atual.
7. Chama `model.predict` se não há cache compatível.
8. Salva resposta e auditoria; só então acessa `df.iloc[window.target]`.
9. Associa cada previsão ao alvo da mesma antecedência e grava CSV/status.

fingerprint e source_fingerprint

`fingerprint(value)` serializa uma estrutura com chaves ordenadas e calcula SHA-256. `source_fingerprint()` inclui caminhos e conteúdo dos arquivos Python de `src`. A procedência combina código, dataset, configuração, identidade do modelo, quantidade de janelas e eventual limite de teste.

O `request_id` inclui também a janela e o texto exato do prompt. Só há reaproveitamento se o identificador coincide e o status anterior é `ok`. A retomada pressupõe que os artefatos locais não foram alterados manualmente; hashes são identificação de procedência, não assinatura de segurança.

write_json

Grava primeiro em um arquivo `.tmp` e usa `replace` para substituir o destino completo. Assim, uma interrupção durante escrita não deixa metade do JSON principal. `allow_nan=False` impede artefatos JSON não padronizados. Essa função não fornece trava entre processos: execute somente uma instância por diretório de saída.

As previsões intermediárias por janela permitem retomar uma execução interrompida mesmo antes de o CSV final existir. O tempo usa `time.monotonic()`, adequado para duração porque não depende de ajustes no relógio de calendário.

12 · Baselines e métricas comparáveis

ChronosForecaster

O construtor carrega `amazon/chronos-2` do cache local, usa CUDA quando disponível e fixa a semente. `predict` fornece um dataframe com identificador de série, data e demanda histórica a `predict_df`. Os parâmetros `prediction_length` e `context_length` refletem o mesmo horizonte e contexto usados no Qwen. `quantile_levels=[0.5]` seleciona a mediana como previsão pontual.

TimesFMForecaster

Carrega `google/timesfm-2.5-200m-pytorch` e compila com contexto arredondado ao próximo múltiplo de 32, por exigência dos patches. Esse arredondamento define capacidade interna; o vetor de observações continua com exatamente 365 ou 565 valores. `max_horizon=128` é capacidade interna, enquanto `forecast(horizon=7, inputs=[...])` pede os sete dias avaliados.

`normalize_inputs=True` delega a normalização da entrada histórica ao modelo; não ajusta estatísticas usando alvos. As demais opções seguem o caminho de inferência configurado no adaptador. O retorno contém previsão pontual e quantis; usamos a primeira.

deterministic_metrics

Métrica	Definição e leitura
MAE	Média de |previsão - observado|. Mesma unidade da demanda; menor é melhor.
RMSE	Raiz da média do erro ao quadrado. Penaliza erros grandes; menor é melhor.
MAPE (%)	100 × média de |erro/observado|; exclui observados zero e informa contagem.
sMAPE (%)	100 × média de 2|erro|/(|observado|+|previsão|); par 0/0 contribui zero.
R^2	1 - soma dos erros² / soma dos desvios do observado em relação à sua média². Pode ser negativo; alvo constante fica indefinido.

`compare(output)` exige todos os modelos, nenhum erro, nenhuma execução limitada, mesmas chaves temporais/alvos e mesma configuração/dataset/código. Não compara um subconjunto conveniente de janelas em que o Qwen acertou o formato. As métricas principais agrupam todos os pontos por cidade; um segundo CSV separa os sete leads.

O tempo registrado por janela exclui a construção inicial do adaptador. No Qwen pode incluir carregamento preguiçoso do servidor; não deve ser tratado como comparação controlada de latência dos motores.

13 · Resultados da execução completa

Os números abaixo foram lidos do CSV produzido pelas inferências desta execução, sem reutilizar as métricas antigas. São 54 janelas em Joinville (378 alvos) e 104 em Guaratuba (728 alvos), por modelo.

Dataset	Modelo	MAE	RMSE	MAPE %	R^2
joinville	qwen	19154.309	22981.845	8.915	-1.496
joinville	chronos	8636.334	10870.152	3.980	0.442
joinville	timesfm	8543.088	10751.578	3.934	0.454
guaratuba	qwen	3546.063	4801.616	28.760	-0.858
guaratuba	chronos	1385.454	1807.227	11.265	0.737
guaratuba	timesfm	1406.671	1842.025	11.175	0.727

Compare modelos dentro da mesma cidade: as escalas de produção diferem entre elas. Um R^2 negativo significa que a soma dos erros quadráticos supera a referência da média dos alvos usada na definição da métrica; não significa erro de parsing.

O protocolo foi definido antes da avaliação. Não houve escolha de prompt, hiperparâmetro ou modelo vencedor para reexecutar sobre o teste após observar essas métricas. Desempenho ruim deve permanecer no relatório.

Dataset	Modelo	Início do teste	Fim do teste	Segundos de inferência
joinville	qwen	2024-06-16	2025-06-28	159.46
joinville	chronos	2024-06-16	2025-06-28	1.63
joinville	timesfm	2024-06-16	2025-06-28	2.34
guaratuba	qwen	2018-01-01	2019-12-29	753.94
guaratuba	chronos	2018-01-01	2019-12-29	1.56
guaratuba	timesfm	2018-01-01	2019-12-29	4.00

Consulte `metrics_by_horizon.csv` para separar o primeiro e o sétimo dia, e `predictions.csv` para examinar eventos individuais. Resultados completos, incluindo sMAPE e tempos médios, permanecem no CSV principal.

14 · Executar e investigar uma janela

Execução completa

```
# Na raiz do projeto; Ollama deve estar ativo.
ollama serve
# Em outro terminal:
bash experiments/run_benchmark.sh
```

O script executa Qwen no ambiente `promptcast`, libera o modelo do Ollama, executa Chronos no ambiente `chronos`, TimesFM no ambiente `timesfm` e compara resultados. Os ambientes existentes não são modificados. A configuração fica em `configs/benchmark.json`.

Executar Qwen isoladamente

```
conda run -n promptcast python -m src.pipelines.forecasting_pipeline --model qwen
# Apenas integração, sem produzir comparação final:
conda run -n promptcast python -m src.pipelines.forecasting_pipeline     --model qwen --limit 1 --
output experiments/smoke
```

O entry point `qwen_forecast` está declarado no `pyproject.toml` para instalação futura. Nesta execução usamos `python -m` para não instalar nada nos ambientes. O trecho `if __name__ == "__main__"` chama o parser quando o módulo é executado, mas não inicia inferência quando outro módulo importa funções para testes.

Exemplo de investigação

1. Abra `experiments/benchmark/guaratuba/prompts/2018-01-01.txt`: os valores devem terminar em 31/12/2017.
2. Abra o JSON de mesmo nome em `guaratuba/qwen/responses`: observe request, attempts, response e predictions.
3. Confira `prompt_eval_count` e `eval_count` no envelope do Ollama.
4. Localize a origem em `guaratuba/qwen/predictions.csv`: sete linhas devem corresponder a sete leads.
5. Consulte `run.json` para digest do modelo, hashes e parâmetros.

Ao mudar código ou configuração, use um novo diretório de saída para conservar o experimento anterior. O cache atual invalida configurações diferentes, mas o mesmo caminho será sobrescrito pela nova execução.

15 · Testes, manutenção e extensões

Os testes não precisam carregar os modelos. Eles verificam propriedades que poderiam invalidar a comparação, complementados por testes reais de uma janela e pela execução completa.

Verificação	Risco coberto
Contexto e datas nas duas fontes	Uma alteração de slicing incluir alvos ou usar horizonte errado.
Mutação de alvos futuros	Mudar o teste modificar o prompt atual.
Few-shot restrito ao prefixo	Usar testes como respostas de demonstrações.
Preenchimento causal	Interpolação usar uma observação futura.
Exclusão de alvo preenchido	Métrica tratar valor imputado como verdade observada.
Parser estrito	Aceitar booleanos, texto, negativos, NaN ou horizonte errado.
Retentativas e auditoria	Perder a resposta bruta ou não registrar falha anterior.
Orçamento do prompt	Truncar contexto histórico silenciosamente.
Métricas conhecidas	Confundir MAPE percentual com fração ou remover linhas desalinhadas.
Validação temporal e datas únicas	Misturar partições ou aceitar duplicatas ambiguamente.

```
conda run -n promptcast python -m unittest discover -s tests -v
```

Como estender sem quebrar o protocolo

Um novo modelo deve receber somente história e devolver exatamente `horizon` números. Uma futura covariável meteorológica exige origem de emissão, relação com cada data prevista e comprovação de disponibilidade anterior à origem. Uma transformação aprendida deve ser ajustada exclusivamente no treino da partição apropriada.

Se for comparar estilos de prompt, reserve uma validação temporal dentro do prefixo de treino. Escolha a configuração nela e congele a escolha antes de medir o teste. Não use o CSV final para escolher a melhor configuração e depois anunciar esse mesmo teste como avaliação independente.

Referências e versões

- Ollama API generate e saídas estruturadas.
- Amazon Chronos — biblioteca instalada 2.3.1; modelo amazon/chronos-2.
- Google TimesFM — biblioteca instalada 2.0.2; modelo TimesFM 2.5 200M.
- Python dataclasses, argparse e argumentos de funções.

A versão da biblioteca TimesFM e a geração do modelo são identificadores diferentes: instalar a distribuição 2.0.2 nesta máquina não significa executar os pesos TimesFM 2.0. O identificador real dos pesos está no adaptador e em `run.json`.

Apêndice A · build_prompt completo

Fonte: `src/data/preprocessing/generate_prompts.py`. O corpo abaixo é extraído diretamente da implementação entregue.

```
def build_prompt(history: pd.DataFrame, horizon: int, *,
                 demonstrations: list[Demonstration] | None = None) -> str:
    """Accept history only, not the complete dataframe or a target dataframe."""
    if horizon < 1 or history.empty:
        raise ValueError('History and positive horizon are required')
    values = history.water_produced.to_numpy(dtype=float)
    if not np.isfinite(values).all():
        raise ValueError('History contains invalid values')
    dates = future_dates(history.timestamp.iloc[-1], horizon)
    parts = [
        'Forecast daily water production in cubic metres.',
        'Use the chronological daily history below to infer level, trend and seasonality.',
        f'Return ONLY a JSON object with key "predictions" and exactly {horizon} nonnegative
numbers.',
        'Do not explain. Do not repeat the historical values as an answer.'
    ]
    for example in demonstrations or []:
        parts.append('Historical example input: ' + format_values(example.history))
        parts.append('Historical example output: ' + json.dumps({'predictions':
list(example.target)}))
    parts.extend([
        f'History: {len(history)} consecutive daily observations, '
        f'from {history.timestamp.iloc[0].date()} to {history.timestamp.iloc[-1].date()}.',
        'Values (oldest to newest): ' + format_values(values),
        'Forecast dates in output order: ' + ','.join(dates.strftime('%Y-%m-%d')),
        f'Output schema: {{"predictions": [number, ...]}}; array length must be {horizon}.'
    ])
    return '\n'.join(parts)
```

Apêndice B · predict completo

Fonte: `src/models/promptcast.py`. A auditoria mantém cada tentativa e só retorna após validar a resposta.

```
def predict(self, history, horizon: int, *, prompt: str) -> list[float]:
        """Send only a prebuilt, target-free prompt; retain every attempted response."""
        config = self.config
        # UTF-8 byte count is a conservative upper bound for byte-level tokenizers.
        # Reserve space for the Ollama model template and the entire generated answer.
        if len(prompt.encode('utf-8')) + config.num_predict + 512 > config.num_ctx:
            raise ValueError('Prompt exceeds conservative context budget; increase num_ctx')
        payload = {'model': config.model, 'prompt': prompt, 'stream': False,
                   'think': False, 'keep_alive': '10m', 'format': prediction_schema(horizon),
                   'options': {'temperature': config.temperature, 'seed': config.seed,
                               'num_ctx': config.num_ctx, 'num_predict': config.num_predict}}
        self.last_audit = {'request': payload, 'attempts': []}
        for attempt in range(config.retries + 1):
            audit = {'attempt': attempt + 1}
            self.last_audit['attempts'].append(audit)
            try:
                answer = post_json(config.base_url + '/api/generate', payload, config.timeout)
                audit['response'] = answer
                if not answer.get('done') or answer.get('done_reason') == 'length':
                    raise ValueError('Incomplete or token-limited Ollama response')
                if answer.get('prompt_eval_count', 0) + answer.get('eval_count', 0) >=
config.num_ctx:
                    raise ValueError('Context limit reached')
                return parse_prediction(answer.get('response', ''), horizon)
            except (ValueError, urllib.error.URLError, TimeoutError) as exc:
                audit['error'] = str(exc)
                if attempt == config.retries:
                    raise
                time.sleep(min(2 ** attempt, 4))
        raise RuntimeError('Unreachable retry state')
```